

EFREI

Mémoire de fin d'année — PGE ING2/ING3 — 2025-2026

Développement d'un outil d'aide à la décision pour la transition électrique d'une flotte de véhicules thermiques

Analyse GPS, scoring comportemental et infrastructure de recharge

Auteur	Sofian EL YAMANI
Promotion	ING2 / ING3
Tuteur Entreprise	Jean-Philippe EDOM
Tuteur Pédagogique	Badr TAJINI
Chargé de Mission Career Center	Orlane VAN DEN MEERSSCHAUT
Entreprise	3C Compétence Cuisine Collective

Année académique 2025–2026

Résumé

Ce mémoire documente le développement d'une application web construite pendant mon alternance chez 3C. L'entreprise dispose d'une centaine de véhicules de service utilisés au quotidien par des techniciens itinérants, tous équipés de boîtiers GPS Masternaut. Le problème de départ était simple : les données GPS étaient là, mais Masternaut ne permettait pas de faire les analyses que la direction voulait faire.

L'application comporte quatre modules. Une **carte interactive** affiche les trajets GPS d'un conducteur sur une période choisie librement, avec une timeline latérale pour naviguer trajet par trajet, visualiser les bornes de recharge IRVE directement sur le parcours, et voir les arrêts intermédiaires. Un **module arrêt moteur tournant** classe les véhicules par durée d'arrêt moteur tournant, estime le carburant gaspillé et les émissions CO₂, et exporte le tout en Excel. Un **dashboard comportemental** classe les conducteurs par style de conduite à partir des indicateurs Masternaut. Enfin, le module le plus complexe : un **scoring EV** où DBSCAN identifie les lieux d'arrêt habituels de chaque conducteur à partir des traces GPS brutes, les croise avec la base nationale des bornes IRVE (data.gouv.fr), et un modèle Gradient Boosting valide la classification en deux niveaux de convertibilité, affinée ensuite en quatre niveaux via une régression logistique multinomiale.

L'application est développée en React 19 sans serveur backend. Un cache local IndexedDB (Dexie) réduit la charge sur l'API Masternaut et assure la conformité RGPD par expiration automatique des données à un an.

Mots-clés : fleet management, GPS, Gradient Boosting, régression logistique, IRVE, arrêt moteur tournant, transition EV, React.

Table des matières

Résumé	1
1 Introduction générale	4
1.1 Contexte et motivation	4
1.2 Problématique et objectifs	5
1.3 Contributions et structure	5
2 Position du problème	6
2.1 L'entreprise et sa flotte	6
2.2 Les quatre besoins à l'origine du projet	6
2.3 Pourquoi la conversion EV est difficile pour cette flotte	11
2.4 Les données et leurs contraintes	12
2.5 Problématique	12
3 État de l'art	13
3.1 Les outils de gestion de flotte existants	13
3.2 Analyse de trajectoires GPS et clustering spatial	14
3.2.1 K-means et ses limites sur des données GPS	14
3.2.2 DBSCAN : clustering par densité pour l'extraction des arrêts habituels	15
3.3 Évaluation de la viabilité EV : approches existantes	15
3.4 La base IRVE comme source de données d'infrastructure	16
4 Étude originale	17
4.1 Architecture technique	17
4.1.1 Base de données locale (IndexedDB via Dexie)	17
4.2 Ingénierie des features	19
4.2.1 Variables de volume	19
4.2.2 Variables de distance	19
4.2.3 Variables de variabilité	20
4.2.4 Variables de profil	20

4.2.5	Variables de rythme	20
4.2.6	Variable temporelle	21
4.2.7	Variables les plus discriminantes	21
4.3	Carte interactive et performance Leaflet	22
4.4	Arrêt moteur tournant : estimation carburant en deux phases	22
4.5	Extraction des arrêts habituels par DBSCAN	23
4.5.1	Extraction des arrêts entre trajets	23
4.5.2	Clustering DBSCAN des points d'arrêt	23
4.6	Le score EV : formule et pondération	24
4.7	Validation par Gradient Boosting avec data augmentation	25
4.7.1	Problème du petit dataset et overfitting	26
4.7.2	Data augmentation gaussienne	26
4.7.3	Résultats	27
4.7.4	Concordance avec le score métier	28
Conclusion		29
Remerciements		31

Chapitre 1

Introduction générale

1.1 Contexte et motivation

Depuis la loi d’orientation des mobilités de 2019, les entreprises doivent renouveler une part croissante de leurs flottes en électrique ou hybride, soit 40 % des nouvelles acquisitions dès 2025 pour les véhicules légers. Ajoutez à ça les engagements RSE et la pression des clients sur l’empreinte carbone, et la transition EV est devenue un sujet qu’on ne peut plus remettre à plus tard.

3C a une flotte d’une centaine de véhicules de service. Les profils de déplacement varient beaucoup d’un technicien à l’autre : certains font des interventions courtes en zone urbaine, d’autres couvrent des secteurs ruraux avec des trajets bien plus longs. Les véhicules des techniciens sont équipés d’un boîtier GPS Masternaut qui remonte les coordonnées, distances, durées et indicateurs de conduite pour chaque trajet. Ces données étaient là, mais personne ne les exploitait vraiment. Les équipes commençaient à se poser des questions concrètes : quels véhicules passent le plus de temps arrêtés avec le moteur qui tourne ? Qui conduit le plus agressivement ? Et surtout : lesquels pourrait-on passer à l’électrique ?

Masternaut permet de sélectionner un conducteur précis et de consulter ses arrêts, mais dès qu’on veut faire quelque chose de plus précis c’est bloqué. Il n’est pas possible de filtrer librement sur les dates souhaitées sans contrainte de période, ni de visualiser les bornes de recharge directement sur le trajet ou de comparer leur distance par rapport aux points d’arrêt habituels. C’est ce manque qui a lancé le projet.

1.2 Problématique et objectifs

La question centrale du projet : comment exploiter les données GPS d’une flotte professionnelle pour produire des analyses par conducteur sur les comportements de conduite, le gaspillage de carburant et la faisabilité d’une conversion EV, là où les outils existants ne permettent pas ce niveau de personnalisation et d’analyse croisée ?

Trois choses à construire pour répondre à ça. Une carte qui rende les trajets lisibles conducteur par conducteur, avec les bornes IRVE intégrées et un filtrage de dates sans limite. Des modules d’analyse sur les données brutes Masternaut : arrêt moteur tournant, comportement, classements. Et un scoring EV qui croise les habitudes GPS de chaque conducteur avec les bornes disponibles, et qui donne une recommandation qu’on peut expliquer sans formation en data science.

1.3 Contributions et structure

La carte apporte un meilleur visuel sur les trajets qu’il n’est pas possible d’avoir dans Masternaut : sélectionner un conducteur précis, visualiser un seul trajet isolé, filtrer librement sur les dates, et comparer directement la distance des bornes de recharge par rapport aux arrêts habituels. Le module arrêt moteur tournant a donné pour la première fois un chiffre sur le carburant gaspillé à l’échelle de la flotte. Le scoring EV croise les arrêts GPS réels avec les bornes IRVE, là où les outils du marché se contentent de comparer la distance à une autonomie théorique.

Ce mémoire est structuré en trois parties : la **position du problème**, l’**état de l’art**, et l’**étude originale** qui détaille les choix techniques, les algorithmes et les résultats.

Chapitre 2

Position du problème

2.1 L’entreprise et sa flotte

3C est une entreprise spécialisée dans l’installation et la maintenance de cuisines professionnelles (équipements de restauration collective et cuisine industrielle). Elle emploie des techniciens itinérants qui utilisent quotidiennement des véhicules de service pour réaliser des interventions chez leurs clients. La flotte comprend une centaine de véhicules thermiques, répartis entre berlines compactes et utilitaires légers. Les véhicules des techniciens sont équipés d’un boîtier GPS Masternaut qui remonte en continu les données de localisation et de conduite vers la plateforme Masternaut Connect.

Masternaut expose trois types de données. Les **trajets** : coordonnées de départ et d’arrivée, heure, distance, identifiant du conducteur. Le **tracking GPS** : trace point par point pour chaque trajet. Et les **indicateurs comportementaux** : accélérations brusques, freinages forts, virages serrés pour cent kilomètres, durées d’arrêt moteur tournant, consommation kilométrique.

Le problème c’est que Masternaut ne laisse pas faire grand-chose de personnalisé avec ces données. Il n’est pas possible de filtrer librement sur les dates souhaitées, de visualiser les bornes de recharge sur le trajet, ni de croiser les habitudes GPS d’un conducteur avec une source externe comme la base des bornes. C’est ça qui a justifié de développer un outil maison.

2.2 Les quatre besoins à l’origine du projet

En discutant avec les équipes, quatre besoins sont ressortis, tous basés sur les mêmes données Masternaut mais pour des usages très différents.

Le premier besoin, c’est la **visualisation des trajets**. Le responsable de flotte voulaient

pouvoir sélectionner un conducteur, choisir une période sans contrainte de durée, et voir ses trajets sur une carte avec les heures, distances et temps d'arrêt entre deux missions, ainsi que les bornes de recharge disponibles sur les parcours.

La figure 2.1 montre la vue d'ensemble d'une journée de travail complète pour un technicien : treize trajets répartis sur 167 km et 4h01 de conduite, chacun tracé dans une couleur distincte. La timeline latérale liste chronologiquement chaque déplacement avec l'heure de départ, l'adresse de départ et d'arrivée, la durée et le temps d'arrêt avant le trajet suivant. Des flèches de direction sont placées sur les tracés pour indiquer le sens de déplacement, et une légende distingue les bornes de recharge selon leur puissance.

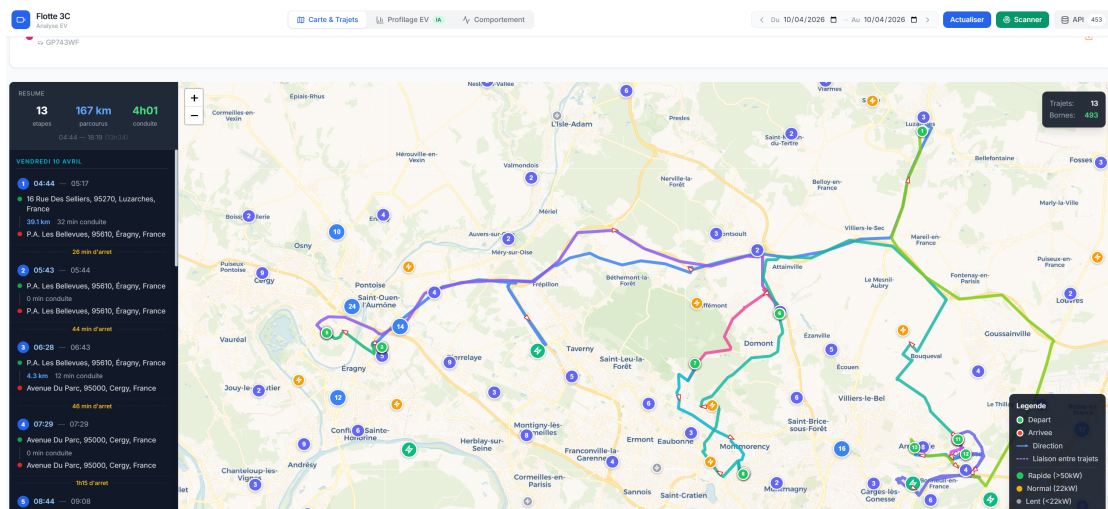


FIGURE 2.1 – Vue d'ensemble : treize trajets d'un technicien sur une journée, avec timeline latérale et bornes IRVE.

Un clic sur un trajet dans la timeline zoome dessus et le passe en mode sélectionné : tracé vert cyan avec un effet de halo, marqueurs de départ (vert) et d'arrivée (rouge) agrandis. Les autres trajets disparaissent pour ne pas surcharger. La figure 2.2 montre ça sur un trajet court à Cergy.

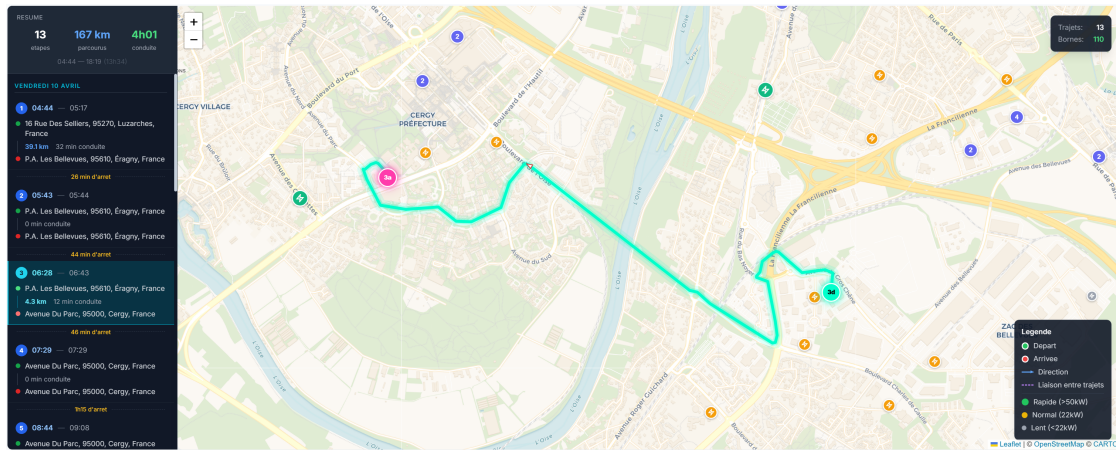


FIGURE 2.2 – Trajet sélectionné en surbrillance fluorescente, avec marqueurs de départ (vert) et d’arrivée (rouge).

Le deuxième besoin, c’est l’**analyse des arrêts moteur tournant**. Un arrêt moteur tournant, c’est le véhicule à l’arrêt avec le moteur qui continue de tourner : pause déjeuner, attente chez un client, embouteillage. Sur l’ensemble de la flotte, même quelques minutes par jour par véhicule ça représente des milliers de litres par an. La direction voulait savoir quels véhicules étaient les plus mauvais, avec une estimation du carburant gaspillé et un export Excel à mettre dans un rapport.

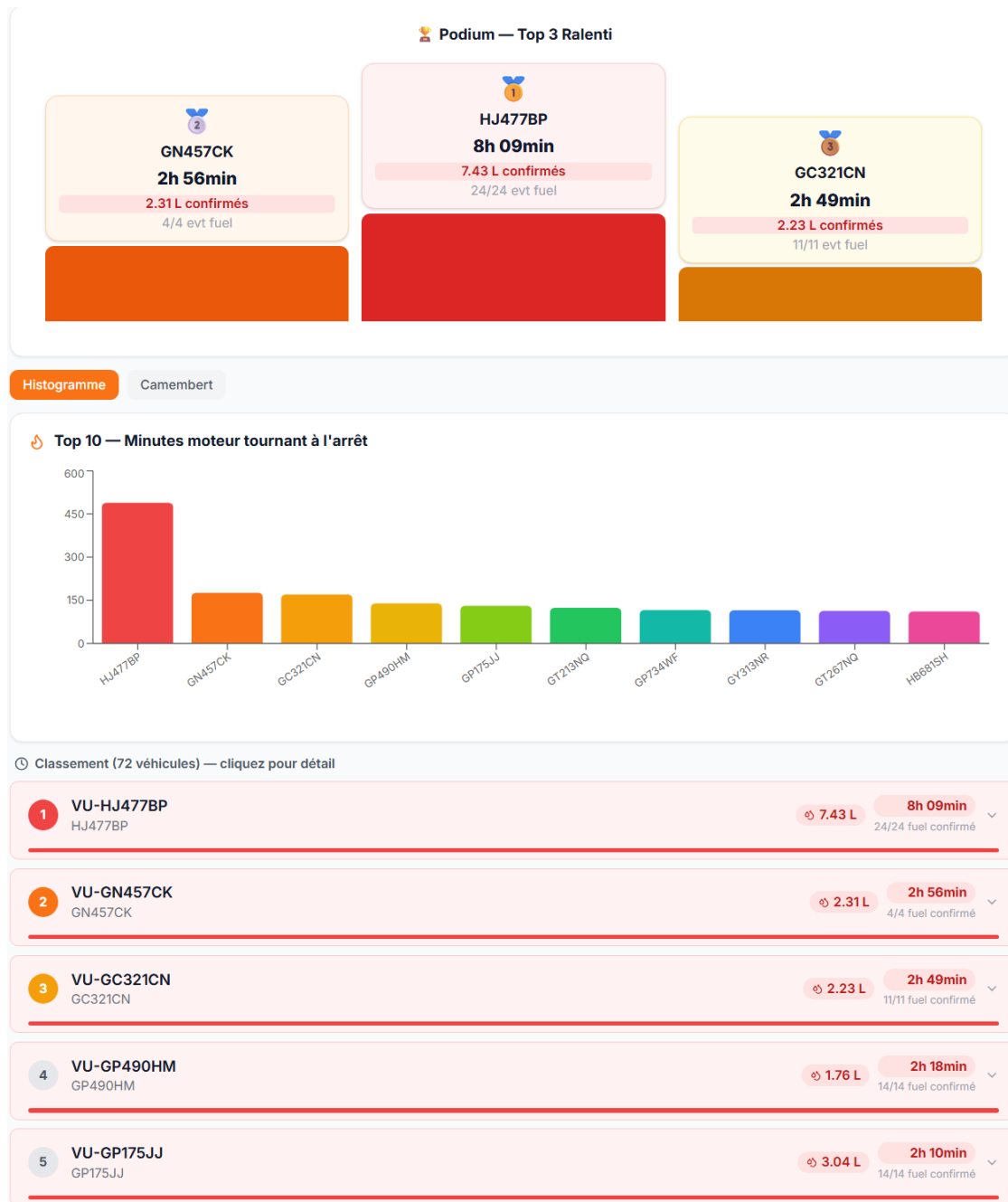


FIGURE 2.3 – Module arrêt moteur tournant : podium des trois véhicules les plus consommateurs (HJ477BP — 8h09min / 7,43 L), histogramme des dix pires et classement complet des 72 véhicules actifs avec carburant et CO₂ estimés.

Le troisième besoin, c'est le **suivi du comportement de conduite**. Masternaut enregistre les accélérations brusques, freinages forts et virages serrés pour cent kilomètres par conducteur. Un conducteur agressif consomme plus, use les pneus et les freins plus vite, et a plus de risques d'accident. L'équipe technique voulait un classement régulier pour savoir qui cibler en priorité pour les formations éco-conduite. Le classement Masternaut natif n'est pas pertinent pour cet usage car il ne prend pas en compte la distance réellement parcourue par chaque conducteur : un technicien qui fait 10 km

par jour sera systématiquement classé premier alors qu'il n'est pas exposé aux mêmes contraintes qu'un technicien qui en fait 200, lequel peut se retrouver dernier pour quelques événements brusques sur un long trajet.

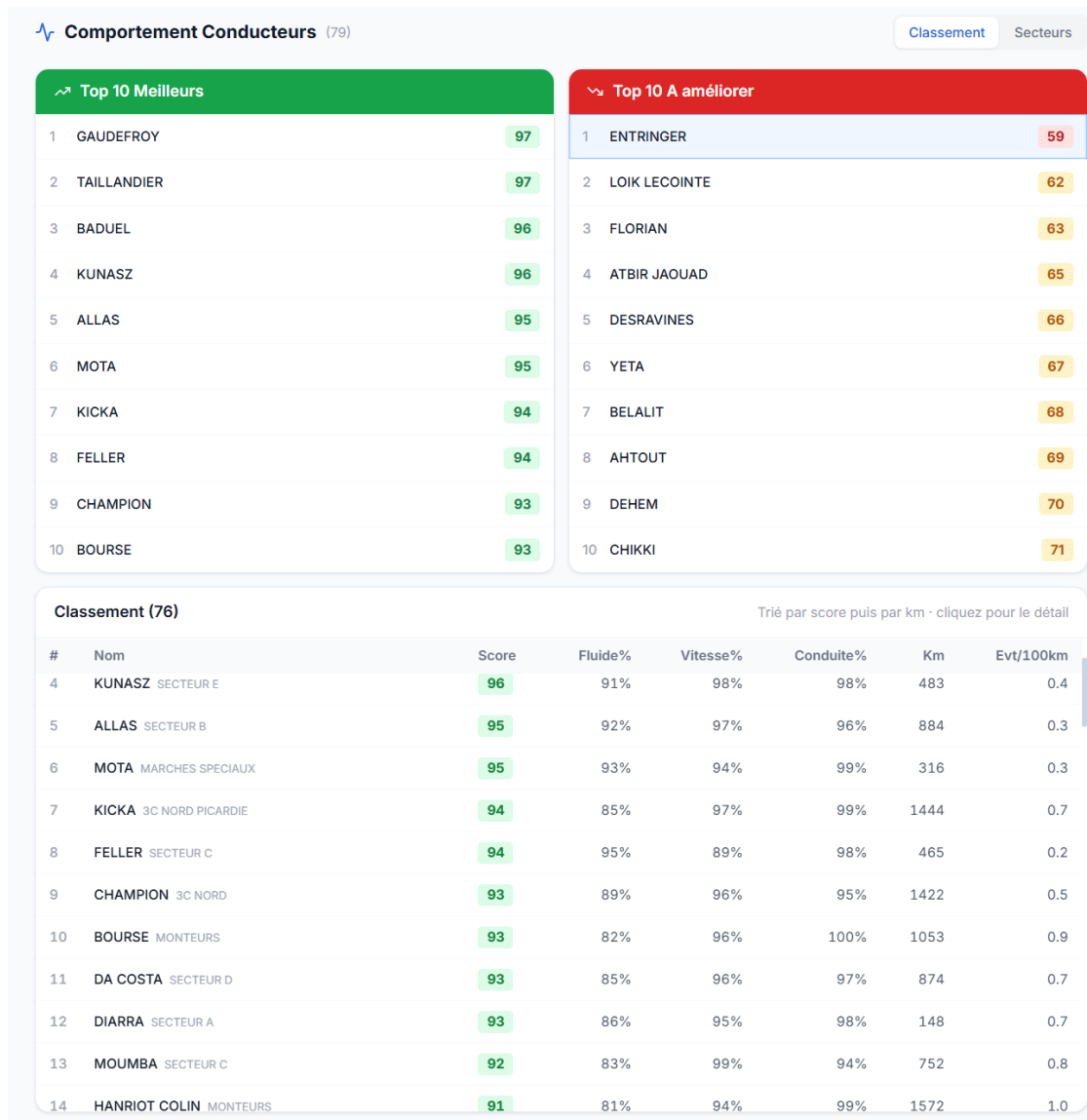


FIGURE 2.4 – Dashboard comportement : classement des 79 conducteurs actifs avec leur score global, top 10 meilleurs (GAUDEFRY 97/100) et top 10 à améliorer (ENTRINGER 59/100), tableau complet avec colonnes Score, Conduite fluide, Respect des vitesses, Efficacité, Km parcourus et Événements/100 km.

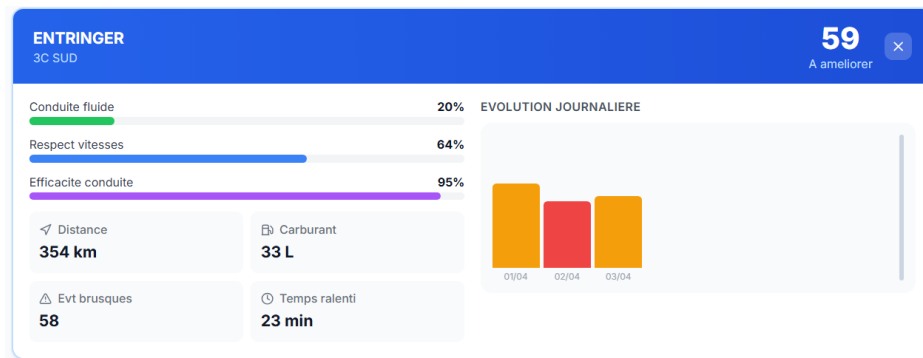


FIGURE 2.5 – Fiche individuelle conducteur : ENTRINGER (3C SUD), score 59 — conduite fluide 20%, respect des vitesses 64%, efficacité conduite 95%, 354 km, 33 L carburant, 58 événements brusques, 23 min d’arrêt moteur tournant, évolution journalière.

Le quatrième besoin, et le plus ambitieux, c’est la **décision de conversion EV**. La direction doit établir un plan de renouvellement pour respecter la loi LOM, mais le budget ne permet pas de tout convertir d’un coup. Il faut donc prioriser : qui peut passer à l’électrique maintenant, qui a besoin d’une borne d’abord, qui est mieux en hybride, qui reste thermique ? Masternaut ne répond pas à ça.

2.3 Pourquoi la conversion EV est difficile pour cette flotte

Ce n’est pas la même chose que convertir une flotte de livraison urbaine ou des commerciaux avec des tournées fixes.

Premier problème : la **variabilité des distances**. Un technicien peut faire 40 km un jour et 250 km le lendemain pour une urgence à l’autre bout du secteur. Raisonner sur la moyenne ne sert à rien : une moyenne de 120 km peut cacher des pics réguliers à 280 km. Et ce sont ces journées-là qui posent problème, pas la moyenne.

Deuxième problème : l’**accès réel aux bornes**. Qu’il y ait des bornes dans un rayon de 3 km ne veut pas dire que le technicien peut s’en servir. Une borne à 3 km d’un endroit où il ne s’arrête jamais, ça ne l’aide pas. En revanche, une borne à 200 m du restaurant où il déjeune tous les jours lui permet de récupérer 10 kWh sans changer ses habitudes. C’est cette différence entre accès théorique et accès pratique que les outils existants ignorent.

Troisième problème : la **disponibilité opérationnelle**. Un technicien en panne de batterie, c’est une intervention client annulée. Le seuil d’acceptation du risque est donc élevé : "ça marche la plupart du temps" ne suffit pas. Il faut que le véhicule tienne

même dans les pires journées habituelles du conducteur.

2.4 Les données et leurs contraintes

Deux sources de données. L’**API Masternaut Connect** en Basic Auth donne accès aux trajets, au tracking GPS, aux événements d’arrêt moteur tournant et aux indicateurs comportementaux. En pratique elle pose deux problèmes. Le rate limiting : si on envoie trop de requêtes d’un coup, l’API coupe, d’où un cache local et un système de retry avec backoff exponentiel. Et la documentation qui ne correspond pas aux vraies réponses JSON : j’ai découvert en cours de projet que plusieurs noms de champs étaient faux dans la doc, ce qui a provoqué des valeurs nulles pendant plusieurs jours dans le dashboard comportemental avant de trouver l’origine du bug.

La **base nationale IRVE** (data.gouv.fr, AFIREV) recense plus de 100 000 bornes en France avec les coordonnées GPS, la puissance, le type de connecteur et l’opérateur. Gratuite, sans clé d’API, mise à jour chaque mois.

Côté RGPD, les données de localisation GPS d’un salarié sont des données personnelles. On a retenu une durée de conservation d’un an, dans les clous pour ce type d’usage professionnel. En pratique ça se traduit par un TTL d’un an sur chaque entrée du cache IndexedDB, avec un nettoyage automatique au démarrage de l’application.

2.5 Problématique

Tout ça converge vers une problématique centrale :

Comment exploiter les données GPS d’une flotte professionnelle pour produire des analyses individualisées sur les comportements de conduite, le gaspillage carburant et la viabilité d’une conversion électrique, en croisant ces données avec l’infrastructure de recharge réelle, là où les outils existants ne permettent pas ce niveau de personnalisation ni de croisement avec des sources externes ?

Derrière ça, trois difficultés techniques bien distinctes. Comment identifier automatiquement les lieux d’arrêt habituels de chaque conducteur à partir de points GPS bruts ? Comment combiner des critères très hétérogènes (distances, arrêts, bornes proches, comportement) dans un score qu’on peut expliquer à quelqu’un sans formation en data science ? Et comment faire tourner tout ça sur la centaine de conducteurs sans serveur dédié, tout en ne conservant pas de données personnelles au-delà d’un an ?

Chapitre 3

État de l’art

3.1 Les outils de gestion de flotte existants

Les plateformes de fleet management comme Masternaut, Geotab ou Fleet Complete fonctionnent toutes sur le même principe : un boîtier embarqué remonte les données GPS et comportementales vers un serveur central, accessible via une interface web et une API. Pour le suivi opérationnel quotidien, ça marche bien : kilométrage, alertes de conduite, gestion des entretiens.

Le problème, c’est dès qu’on sort de l’usage standard. Il n’est pas possible de filtrer les données sur une période librement choisie sans contrainte de durée, et surtout il n’y a aucun moyen de visualiser les bornes de recharge directement sur les itinéraires ou de croiser les habitudes GPS d’un conducteur avec les bornes proches de ses arrêts habituels.

Des outils spécialisés en viabilité EV existent aussi, comme le EVSA de Geotab ou Stable Auto. Leur approche se résume à comparer la distance journalière à l’autonomie du modèle EV ciblé. C’est utile, mais statique : il ne tient pas compte de là où le conducteur s’arrête ni des bornes qui sont réellement accessibles sur ses itinéraires. C’est exactement ce que ce projet cherche à faire différemment.

Concrètement, voilà ce que Masternaut ne permet pas de faire. Sur la carte d’abord : il n’y a pas de vue permettant de filtrer librement sur les dates souhaitées, de sélectionner et isoler un seul trajet, ni de visualiser les bornes IRVE directement sur le parcours avec comparaison de leur distance par rapport aux arrêts. Sur les arrêts moteur tournant : les données brutes sont dans l’API, mais Masternaut n’en fait aucun classement ni aucune estimation carburant : le module le fait, avec l’export Excel que les équipes utilisent directement. Sur le comportement : le classement Masternaut n’est pas pertinent car il ne prend pas en compte la distance réellement parcourue par conducteur, ce qui

avantage systématiquement les petits rouleurs sur les critères bruts d'événements. Enfin le scoring EV est entièrement absent de Masternaut : croiser les arrêts GPS réels de chaque conducteur avec les bornes IRVE disponibles, calculer un score sur 100 et sortir une recommandation en quatre niveaux : tout ça n'existe pas chez eux, même dans les modules payants.

3.2 Analyse de trajectoires GPS et clustering spatial

Extraire des informations utiles depuis des traces GPS brutes, c'est un problème qu'on retrouve dans beaucoup de contextes : livraison, mobilité urbaine, flotte professionnelle. La vraie difficulté, c'est d'identifier automatiquement les lieux importants (domicile, dépôt, clients habituels) dans une série de points GPS. Ces lieux apparaissent dans les données comme des concentrations de points presque immobiles.

3.2.1 K-means et ses limites sur des données GPS

Le premier algorithme testé a été **K-means**. L'avantage c'est la simplicité : il partitionne un nuage de points en k groupes en minimisant la variance intra-cluster. Sauf que sur les données GPS de cette flotte, ça ne donnait pas de bons résultats.

Le premier problème, c'est la **taille du dataset**. Sur la période analysée, certains conducteurs n'ont que quelques dizaines de points d'arrêt, ce qui est trop peu pour que K-means produise des clusters stables. Le second problème, c'est la **pertinence des clusters obtenus**. K-means suppose des clusters à peu près ronds et de taille similaire, ce qui ne correspond pas à la réalité des arrêts GPS, où on a des points concentrés sur un parking et d'autres dispersés sur plusieurs centaines de mètres le long d'une rue. Il faut aussi fixer k à l'avance, alors qu'on ne sait pas combien de lieux d'arrêt habituels a un conducteur : certains en ont cinq, d'autres trente.

Pour trouver le meilleur k , j'ai calculé le **coefficient de silhouette** pour différentes valeurs. Ce coefficient mesure si chaque point est mieux dans son cluster que dans le voisin : proche de 1 c'est bon, proche de 0 c'est flou, négatif c'est mal classé. Le meilleur score obtenu était 0,38, ce qui est insuffisant. Sur la carte, les frontières de clusters coupaient des zones géographiques évidentes un même dépôt réparti sur deux clusters, des rues différentes regroupées ensemble. K-means n'était pas adapté.

3.2.2 DBSCAN : clustering par densité pour l'extraction des arrêts habituels

Pour l'extraction des lieux d'arrêt habituels à partir des traces GPS, j'ai utilisé **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*). L'idée est différente : au lieu de minimiser une variance, l'algorithme cherche des zones denses dans les données et met de côté les points isolés comme du bruit. Deux paramètres suffisent : ε le rayon de voisinage, et *minPts* le nombre minimum de points pour qu'une zone soit considérée dense.

DBSCAN a plusieurs avantages concrets sur K-means pour ce cas. Pas besoin de fixer k à l'avance : le nombre de clusters émerge des données. Il gère les formes quelconques, ce qui colle mieux à la réalité des arrêts GPS (un parking en L, une zone industrielle allongée). Les points isolés sont traités comme du bruit plutôt que forcés dans un cluster. Et avec la distance de Haversine, le paramètre ε s'exprime directement en mètres, ce qui le rend facile à calibrer intuitivement.

DBSCAN joue ici un rôle de **pré-traitement géographique** : il transforme une liste de coordonnées brutes en une liste de lieux significatifs, qui servent ensuite à calculer les features du modèle de classification EV. C'est le **Gradient Boosting** qui constitue l'algorithme de classification principal, détaillé dans l'étude originale.

Après quelques tests, j'ai fixé $\varepsilon = 200$ m et *minPts* = 2. 200 m c'est à peu près un pâté de maisons, assez large pour regrouper des arrêts sur un même site, assez serré pour distinguer deux clients voisins.

3.3 Évaluation de la viabilité EV : approches existantes

La question de savoir si un véhicule thermique peut être remplacé par un électrique a été pas mal étudiée. On trouve globalement deux façons d'aborder le problème.

La première, c'est les **règles métier simples** : on fixe des seuils, si la distance journalière dépasse rarement 200 km et qu'il y a accès à une borne au dépôt, c'est convertible. Simple à expliquer, mais ça rate les interactions entre critères. Un véhicule qui fait 150 km par jour en zone rurale sans borne va être traité comme un urbain qui passe devant dix bornes par trajet : ce n'est pas la même situation.

La deuxième approche utilise des **modèles de classification** sur plusieurs critères à la fois. Ces modèles montrent que la distance seule ne suffit pas : les habitudes d'arrêt sont au moins aussi importantes. Des modèles plus complexes comme les random forests ou

les réseaux de neurones sur des features GPS enrichies donnent de bons résultats, mais sont difficiles à expliquer sans formation en machine learning.

Pour ce projet j'ai choisi le **Gradient Boosting** comme modèle principal de validation, complété par une **régression logistique multinomiale** pour la classification en quatre niveaux : elle sort des probabilités par classe, ses coefficients sont lisibles directement comme des poids, et elle tient la route même avec une centaine de profils seulement. Quatre classes de sortie (EV Ready, EV Faisable, Hybride recommandé, Thermique à conserver) plutôt qu'un simple oui/non, pour donner assez de nuance sans prétendre à une précision qu'on n'a pas.

3.4 La base IRVE comme source de données d'infrastructure

Pour croiser les arrêts GPS avec les bornes disponibles, il faut une base de données à jour. En France, c'est la **base nationale IRVE** (Infrastructure de Recharge pour Véhicules Électriques), publiée sur data.gouv.fr par le Ministère de la Transition Écologique avec l'AFIREV. En 2025 elle recense plus de cent mille points de charge, avec pour chaque station les coordonnées GPS, la puissance, le type de connecteur et l'opérateur.

Gratuit, sans clé d'API, mis à jour chaque mois : c'est la source la plus facile à utiliser pour ce type de projet. Le défaut c'est que la base n'est pas parfaitement à jour : une borne listée peut être hors service, et une borne récente peut ne pas encore y être. Dans ce contexte c'est acceptable — on évalue des habitudes sur des mois de données GPS, pas un itinéraire en temps réel.

Chapitre 4

Étude originale

4.1 Architecture technique

L'application est une SPA React 19 compilée avec Vite, sans serveur backend. Les requêtes vers Masternaut partent directement du navigateur. Toutes les données récupérées sont stockées localement dans IndexedDB via Dexie 4, avec un TTL d'un an par entrée, ce qui assure la conformité RGPD (nettoyage automatique au démarrage) et évite de ré-interroger l'API à chaque consultation.

Le retry sur l'API est géré avec deux tentatives et backoff exponentiel. Les erreurs 400/401/403/404 sont ignorées sans retry (erreurs définitives). En cas de 429, l'application lit l'en-tête **Retry-After** et attend exactement le délai indiqué avant de réessayer. Sans ce mécanisme, le scan de la centaine de conducteurs en une seule opération échouait systématiquement à mi-parcours.

Un problème concret apparu à l'intégration : plusieurs noms de champs dans les réponses JSON Masternaut ne correspondent pas à la documentation officielle. Le champ de distance s'appelle `distanceKm` (et non `distancesKm`), le score de conduite fluide est `smootherHarshPer100km` (sans *s*), et le CO₂ total est en minuscule : `totalCo2Kg`. Ces divergences ont causé des valeurs nulles dans le dashboard comportemental pendant plusieurs jours, et n'ont été détectées qu'en inspectant les réponses brutes dans les DevTools du navigateur.

4.1.1 Base de données locale (IndexedDB via Dexie)

Sans serveur backend, il fallait quand même persister les données entre les sessions. `localStorage` était exclu d'emblée : il est limité à 5 Mo et ne stocke que des chaînes, ce qui oblige à sérialiser/désérialiser du JSON à chaque accès. IndexedDB est l'alternative native du navigateur : pas de limite de taille pratique, stockage d'objets structurés, et

accès par index sans charger toute la table.

J’ai utilisé **Dexie 4** comme wrapper, qui remplace l’API callbacks natifs d’IndexedDB par des Promises et une syntaxe proche d’un ORM. Le schéma définit huit tables :

- **driverJourneys** / **driverTracking** : trajets et traces GPS par conducteur, indexés sur **driverId**
- **journeys** / **tracking** : données globales par plage de dates, indexées sur une **cacheKey**
- **stations** : les 100 000+ bornes IRVE, rechargées une fois par mois maximum
- **evAnalysis** : scores EV calculés par conducteur
- **behaviorData** : données comportementales (arrêt moteur tournant, CO₂, classements)
- **general** : table de fallback pour les autres requêtes

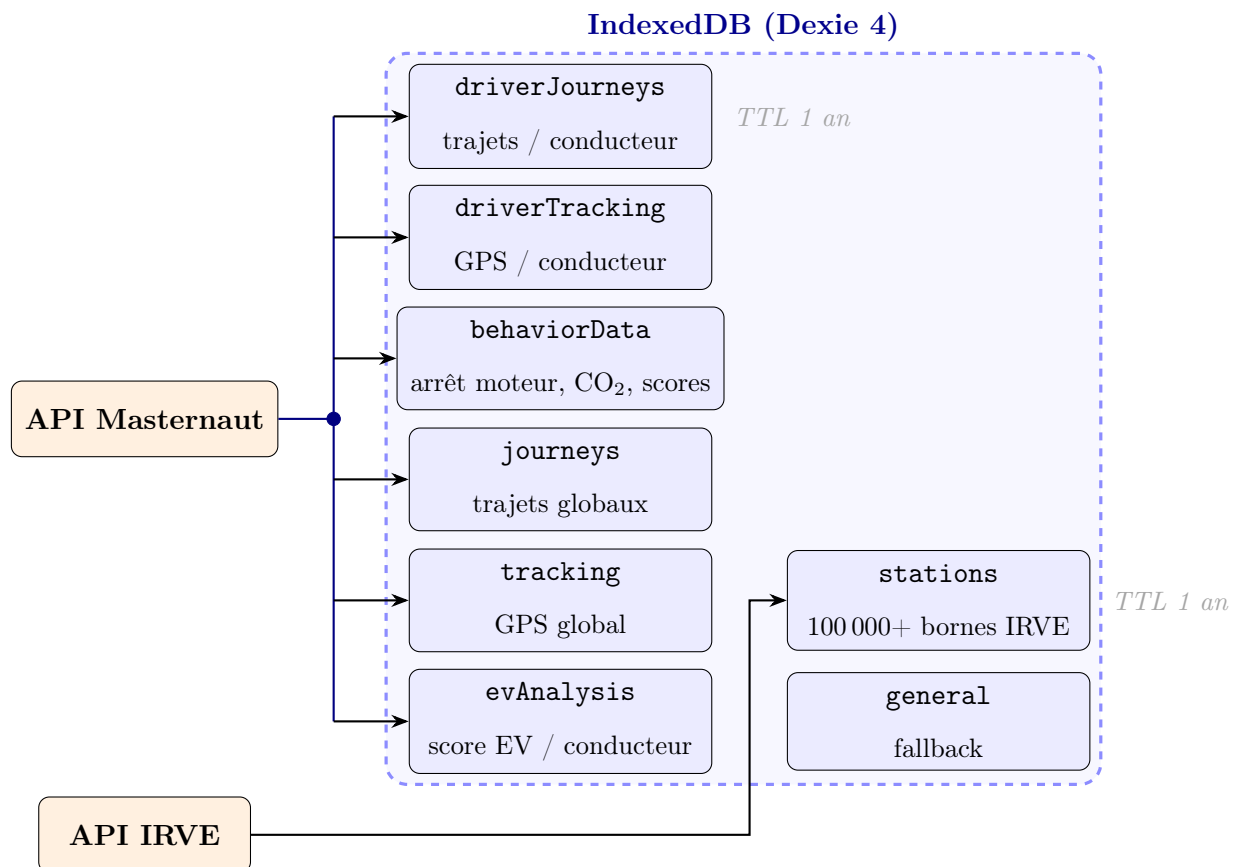


FIGURE 4.1 – Schéma de la base de données locale IndexedDB, composée de 8 tables Dexie alimentées par l’API Masternaut et l’API IRVE. Chaque entrée expire automatiquement après 1 an (RGPD).

Le routage vers la bonne table se fait par préfixe de clé : une clé qui commence par **driver_journeys_** va dans **driverJourneys**, une clé **behavior_** va dans **behaviorData**, etc. Chaque entrée stocke un **createdAt** (timestamp Unix). La vérification d’expiration

se fait à la lecture : si `Date.now() - createdAt > 365 jours`, l'entrée est supprimée et on retourne `null`, ce qui déclenche un rafraîchissement API. Un nettoyage en masse (`cleanupExpiredData`) tourne au démarrage de l'application pour purger tout ce qui a dépassé le TTL RGPD, sans attendre que chaque entrée soit consultée.

En pratique, ça évite de ré-interroger Masternaut à chaque ouverture de l'application. Charger les données comportementales de la centaine de conducteurs prend 3 à 4 minutes la première fois ; avec le cache, c'est instantané les fois suivantes. L'UI expose deux boutons de vidage : un pour les données comportement/arrêt moteur tournant (réponse aux corrections de bugs), un autre qui efface tout (réinitialisation complète). Ces boutons ont servi plusieurs fois pendant le développement quand une correction de bug impliquait de recalculer des données déjà mises en cache.

4.2 Ingénierie des features

Le modèle de classification prend en entrée douze variables calculées par conducteur à partir des trajets Masternaut. Toutes sont calculées sur la même fenêtre temporelle (mars 2025, 31 jours) et uniquement sur les trajets avec une distance positive. C'est cette étape de feature engineering qui transforme une liste de trajets bruts en quelque chose qu'un algorithme peut traiter.

4.2.1 Variables de volume

n_journeys : nombre de trajets. Nombre total de trajets sur la période. Ça dit simplement à quel point le véhicule est utilisé. Un conducteur avec très peu de trajets (moins de 5) n'a pas assez de données pour être classé de façon fiable, donc on le filtre.

active_days : jours actifs. Nombre de jours avec au moins un trajet. Combiné à **n_journeys**, ça donne **trips_per_day**. 40 trajets en 8 jours actifs, c'est un profil très différent de 40 trajets répartis sur 20 jours.

4.2.2 Variables de distance

avg_km : distance moyenne par trajet. La distance moyenne par trajet. Intuitive, mais trompeuse : une moyenne de 80 km peut très bien cacher des journées à 250 km. On la garde dans le modèle mais elle n'est pas ce qui décide du classement.

max_km : distance maximale. La plus grande distance faite sur toute la période. C'est la journée la plus dure que ce conducteur ait eu, et un EV doit tenir même ce jour-là. Le Gradient Boosting en a fait la feature la plus importante du modèle (0,321),

de loin. En pratique, en dessous de 250 km de max, le conducteur est presque toujours EV Ready.

p95_km : 95^e percentile journalier. Le conducteur reste en dessous de cette valeur 95 % du temps. Là où max_km peut exploser à cause d'un seul trajet exceptionnel, p95_km filtre ce bruit : si un technicien a fait une fois 400 km pour remplacer un collègue, ça ne devrait pas plomber son score. C'est cette variable qui pilote S_{dist} dans le score métier.

4.2.3 Variables de variabilité

std_km : écart-type des distances. L'écart-type des distances journalières. Une valeur élevée veut dire que les journées sont très variables, ce qui rend la planification de la recharge difficile. Quatrième variable par importance (0,071), complémentaire à max_km.

cv_km : coefficient de variation. $CV = \sigma/\mu$ (écart-type divisé par la moyenne). Normalise l'écart-type par la moyenne, ce qui le rend comparable entre conducteurs aux distances très différentes. Un conducteur qui fait en moyenne 50 km avec un écart-type de 40 km n'a pas le même profil d'irrégularité qu'un conducteur à 200 km de moyenne avec le même écart-type. Un $CV > 2$ indique une irrégularité structurelle que ni max_km ni p95_km seuls ne suffisent à détecter.

4.2.4 Variables de profil

pct_long : part des trajets longs. Part des trajets au-dessus de 150 km. Un conducteur à 20 % de trajets longs aura régulièrement besoin de recharger en cours de route, ce qui suppose une borne disponible sur son itinéraire, pas juste au dépôt.

pct_short : part des trajets courts. Part des trajets sous 30 km. Un conducteur qui fait essentiellement des courtes missions n'a aucun problème d'autonomie. Le modèle l'utilise comme signal positif : c'est la 5^e feature par importance (0,061).

4.2.5 Variables de rythme

trips_per_day : trajets par jour actif. Nombre moyen de trajets par jour actif. Beaucoup de trajets courts dans la journée (5 à 8 interventions locales), c'est typiquement un profil urbain très compatible avec un EV. Peu de trajets mais longs, c'est l'inverse.

avg_duration_min : durée moyenne d'un trajet. Durée moyenne d'un trajet en minutes (calculée depuis `totalTimeDrivenSeconds` dans l'API). Corrélée à la distance mais pas identique : même kilométrage, durée très différente selon qu'on est en zone urbaine ou sur route dégagée. C'est la 3^e feature par importance (0,104), parce qu'elle capte indirectement le type de territoire : urbain dense ou rural rapide.

4.2.6 Variable temporelle

pct_morning : part des trajets matinaux. Part des trajets démarrés avant 12h. Quelqu'un qui commence tôt chaque matin peut recharger au dépôt la nuit et repartir avec une batterie pleine : c'est le scénario le plus simple pour un EV. Celui qui travaille surtout l'après-midi ou en soirée a moins de marge.

4.2.7 Variables les plus discriminantes

La figure 4.2 montre les importances calculées par le Gradient Boosting sur les 12 features. L'ordre est logique, mais pas forcément celui qu'on aurait prédit.

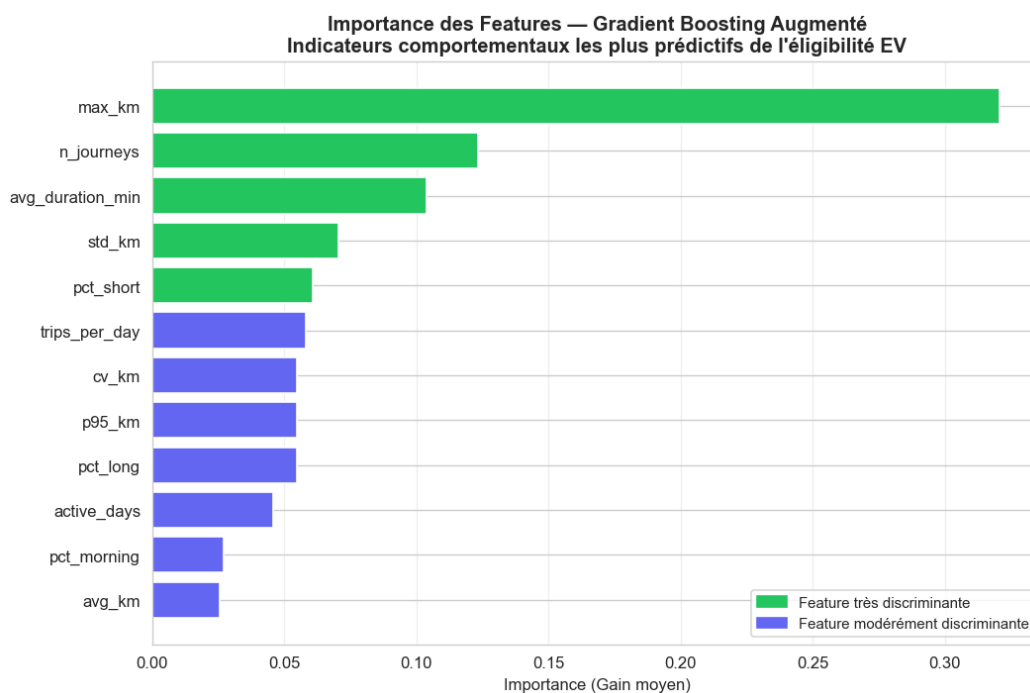


FIGURE 4.2 – Importance des features, modèle Gradient Boosting augmenté. En vert les features très discriminantes (top 5), en violet les secondaires. **max_km** domine largement.

max_km ressort très largement en tête à 0,321, plus du double de la variable suivante. C'est cohérent avec le raisonnement derrière le score métier : ce qui bloque la conversion d'un conducteur, ce n'est pas sa distance moyenne, c'est sa pire journée. En dessous de 250 km de max, le modèle classe presque toujours EV Ready.

`p95_km` ne fait pas partie du top 5, mais il reste utile. Il capte les journées difficiles récurrentes là où `max_km` peut être tiré vers le haut par un seul trajet exceptionnel. Les deux variables sont corrélées, mais pas identiques : c’est pour ça qu’on garde les deux.

`cv_km` sert à filtrer les profils vraiment imprévisibles. Deux conducteurs avec le même `p95_km` peuvent avoir des comportements très différents si l’un est régulier et l’autre fait des journées à 30 km un jour et 300 km le lendemain. Cette irrégularité ne se voit pas dans les autres features seules.

4.3 Carte interactive et performance Leaflet

La carte affiche les trajets d’un conducteur sur 7 jours max, avec une timeline latérale synchronisée. Le vrai problème technique ici c’est la performance : une journée de technicien c’est facilement 10 trajets $\times$ 200 points GPS, soit 2000 éléments à afficher d’un coup.

Deux optimisations concrètes. Chaque tracé est **simplifié à 200 points maximum**, en ne conservant que les points qui représentent le mieux la forme du trajet. Et tous les tracés non sélectionnés passent par le **moteur Canvas** de Leaflet plutôt que SVG : Canvas rend tout dans un seul bitmap sans toucher au DOM, là où SVG crée un nœud par segment et ralentit vite.

```
// Renderer canvas partage (trajets normaux)
const sharedRenderer = L.canvas({ padding: 0.5 });
// Renderer SVG pour le neon (supporte les effets visuels)
const svgRenderer = L.svg({ padding: 0.5 });
```

FIGURE 4.3 – Déclaration des deux renderers Leaflet : Canvas pour les tracés normaux, SVG pour le trajet sélectionné avec effet néon.

Les angles des flèches directionnelles sont précalculés une fois et mis en cache, et non recalculés à chaque zoom. Les 100k+ bornes IRVE passent par `leaflet.markercluster` qui regroupe les marqueurs à faible zoom et les sépare au fur et à mesure qu’on zoome.

4.4 Arrêt moteur tournant : estimation carburant en deux phases

Le module charge les données en deux passes. D’abord l’endpoint arrêt moteur tournant, qui donne immédiatement le classement par durée. Ensuite chaque événement est croisé avec la consommation kilométrique du véhicule pour estimer le carburant gaspillé :

$$\text{carburant (L)} = \frac{\text{conso}_{100\text{km}}}{100} \times 10 \times \frac{t_{\text{AMT}}}{60}$$

Le 10 km/h est une convention pour convertir la conso kilométrique en conso horaire à l'arrêt moteur tournant. C'est une approximation (l'incertitude réelle est autour de $\pm 20\%$ selon le moteur et la température), mais ça suffit pour classer les véhicules entre eux. Le CO_2 est calculé avec le facteur standard essence : 2,68 kg/L.

La figure 2.3 montre le résultat : HJ477BP en tête avec 8h09 d'arrêt moteur tournant et 7,43 L estimés sur un mois. C'est la première fois que la direction avait un chiffre global sur le carburant gaspillé à l'échelle de toute la flotte.

4.5 Extraction des arrêts habituels par DBSCAN

C'est le cœur du pipeline EV. Pour savoir si un conducteur peut recharger sur ses lieux d'arrêt habituels, il faut d'abord identifier ces lieux à partir des données GPS brutes.

4.5.1 Extraction des arrêts entre trajets

Un arrêt est défini comme le gap entre la fin d'un trajet et le début du trajet suivant, à condition que :

- les deux trajets soient **sur la même journée** (pour ne pas confondre avec le parking de nuit)
- le gap soit entre **5 et 480 minutes** (en dessous de 5 min c'est du bruit GPS, au-delà c'est une fin de journée)

La coordonnée retenue pour l'arrêt, c'est la position de fin du trajet précédent (`endCoordinate`). Pour le dépôt, une logique séparée utilise la fin du dernier trajet de la journée, là où le véhicule est rendu le soir. Si le lendemain matin le premier départ est à plus de 300 m, on ajoute ce point comme deuxième signal de dépôt, pour les cas où le véhicule n'est pas toujours garé au même endroit.

4.5.2 Clustering DBSCAN des points d'arrêt

Les points d'arrêt sont ensuite regroupés par DBSCAN avec $\varepsilon = 0,2$ km (rayon de voisinage) et $\text{minPts} = 2$ (nombre minimum de points pour former un cluster). La distance entre deux points est calculée avec la formule de Haversine :

$$d = 2R \arctan 2(\sqrt{a}, \sqrt{1-a}), \quad a = \sin^2 \frac{\Delta\varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta\lambda}{2}$$

où φ désigne la latitude, λ la longitude, Δ la différence entre les deux points, et $R = 6371$ km le rayon terrestre.

L'implémentation calcule en amont la matrice de voisinage complète, de complexité $\mathcal{O}(n^2)$, acceptable pour $n < 2000$ arrêts par conducteur.

```
// Pré-calcul des voisins pour chaque point (O(n²) - acceptable pour < 2000 arrêts)
const neighborsCache = new Array(n);
for (let i = 0; i < n; i++) {
  neighborsCache[i] = [];
  for (let j = 0; j < n; j++) {
    if (i === j) continue;
    if (haversineKm(stops[i].latitude, stops[i].longitude, stops[j].latitude, stops[j].longitude) <= epsilonKm) {
      neighborsCache[i].push(j);
    }
  }
}
```

FIGURE 4.4 – Précalcul du cache de voisinage : pour chaque point, on stocke la liste des points à moins de ε km via Haversine.

Chaque cluster qui sort de là correspond à un lieu où le conducteur s'arrête régulièrement. Pour chacun, on cherche la borne IRVE la plus proche : en dessous de 500 m, c'est une opportunité de recharge réaliste. Ces clusters servent ensuite de features d'entrée pour le scoring EV et le modèle Gradient Boosting.

4.6 Le score EV : formule et pondération

Le score EV est un nombre entre 0 et 100, combinaison pondérée de quatre sous-scores :

$$S = 0,35 \cdot S_{\text{dist}} + 0,35 \cdot S_{\text{borne}} + 0,20 \cdot S_{\text{arrêt}} + 0,10 \cdot S_{\text{régul}}$$

S_{dist} : autonomie suffisante (35%). Calculé sur le **95^e percentile** de la distance journalière, pas la moyenne. La moyenne peut être de 80 km et le conducteur faire 250 km une fois par semaine : c'est cette journée qui pose problème. Plafond à 450 km (autonomie max d'un EV haute gamme). En dessous de 150 km de p95, score à 100. Au-dessus de 450 km, score à 0.

$$S_{\text{dist}} = \max\left(0, \frac{450 - \max(p95, 150)}{300} \times 100\right)$$

S_{borne} : accès réel à la recharge (35%). Part des clusters d'arrêt avec une borne IRVE à moins de 500 m : $S_{\text{borne}} = (n_{\text{clusters proches borne}} / n_{\text{clusters total}}) \times 100$. C'est le critère le plus discriminant en pratique. Si tous les arrêts habituels d'un conducteur sont loin de toute borne, il ne peut pas recharger sans modifier ses habitudes de travail.

$S_{\text{arrêt}}$: **temps disponible pour recharger (20%)**. Durée moyenne des arrêts diurnes ramenée à 90 minutes, soit le temps pour récupérer environ 25 kWh sur une borne 22 kW. 90 min d'arrêt moyen = score 100. 45 min = score 50.

$S_{\text{régul}}$: **régularité des distances (10%)**. Coefficient de variation : $CV = \sigma/\mu$ (écart-type / moyenne). Un conducteur avec $CV > 2$ a des journées tellement imprévisibles qu'on ne peut pas garantir qu'un EV tiendra même si la moyenne est bonne.

Score ≥ 75 : le conducteur est classé **EV Ready**. Ensuite la régression logistique multinomiale affine ça en quatre niveaux (EV Ready / EV Faisable / Hybride recommandé / Thermique à conserver) sur 15 variables, avec des coefficients calibrés manuellement. La sortie softmax donne une probabilité par classe, permettant de voir non seulement où le conducteur est classé, mais aussi avec quelle certitude.

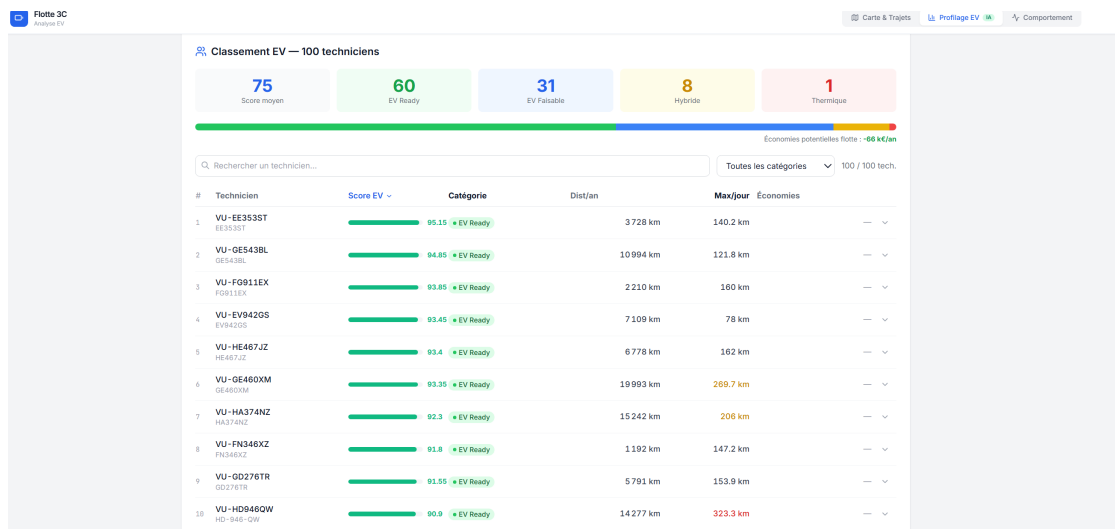


FIGURE 4.5 – Interface du scoring EV : chaque conducteur reçoit un score sur 100 et une classification en quatre niveaux. Les barres de couleur indiquent la contribution de chaque composante du score (S_{dist} , S_{borne} , $S_{\text{arrêt}}$, $S_{\text{régul}}$).

4.7 Validation par Gradient Boosting avec data augmentation

Pour vérifier que les coefficients calibrés à la main ne sont pas juste le fruit de l'intuition, j'ai entraîné un second modèle dans un notebook Python : un **Gradient Boosting** sur les mêmes données de la flotte.

Le Gradient Boosting, c'est un ensemble d'arbres de décision entraînés séquentiellement : chaque arbre corrige les erreurs du précédent, plutôt que de travailler indépendamment comme dans un Random Forest. Concrètement, à chaque étape on entraîne un arbre

sur les résidus du modèle courant, puis on l'ajoute avec un poids réduit (le *learning rate*). Ça converge lentement mais précisément, sans sur-ajuster d'un coup.

Pourquoi pas une régression logistique ou un arbre unique ? Sur un dataset aussi petit (92 conducteurs), un arbre seul overfit immédiatement et une SVM demande une mise à l'échelle parfaite des features. Le Gradient Boosting a deux avantages ici : il n'a pas besoin de normalisation des variables (les arbres découpent par seuils, pas par distances), et il produit directement les importances de features, ce qui m'intéressait autant que la prédiction elle-même pour valider le score métier. J'ai aussi testé le Random Forest, mais le GB donnait des résultats un peu plus stables en cross-validation.

Les hyperparamètres retenus après quelques essais : `n_estimators=200`, `max_depth=3`, `learning_rate=0.05`, `subsample=0.8`, `min_samples_leaf=3`. La profondeur 3 limite la complexité de chaque arbre, le subsample 0,8 introduit un peu de stochasticité pour éviter l'overfitting, et le learning rate bas avec beaucoup d'estimateurs est le réglage classique pour ce type d'algo. Ces valeurs ne sont pas venues d'une grid search exhaustive : avec 92 points, ça aurait été du surréglage. Elles viennent d'essais manuels guidés par la variance en cross-validation.

4.7.1 Problème du petit dataset et overfitting

Sur les 154 conducteurs configurés, 92 avaient assez de trajets pour être analysés (seuil : 5 trajets minimum). Split stratifié 80/20 : 73 en train, 19 en test. Distribution : 53 EV Ready (57,6 %) et 39 Non EV Ready (42,4 %).

Le premier problème était la **taille réduite du dataset** : avec seulement 73 conducteurs en entraînement, le modèle apprenait par cœur plutôt que de généraliser, avec un écart train/test de **31,6 %**. L'accuracy en cross-validation tombait à 67,3 % $\pm$ 9,3 %, avec une instabilité qui rendait les résultats peu exploitables.

4.7.2 Data augmentation gaussienne

Impossible de collecter plus de données réelles : la flotte a ce qu'elle a. La solution est de générer des conducteurs synthétiques. Pour chaque classe, on calcule la moyenne μ_c (centroïde de la classe) et la covariance Σ_c (dispersion multidimensionnelle) des vrais conducteurs, puis on tire 120 profils fictifs :

$$x^{(\text{synth})} \sim \mathcal{N}(\mu_c, \Sigma_c) \quad (\text{loi normale multivariée})$$

En plus, chaque conducteur réel est dupliqué deux fois avec un petit bruit gaussien ($\sigma = 0,15$, soit l'écart-type du bruit ajouté dans l'espace standardisé), pour simuler la

variabilité naturelle des mesures. Résultat final :

TABLE 4.1 – Composition du dataset d’entraînement après augmentation

Source	Conducteurs	Part
Données réelles	73	15,9 %
Synthétiques gaussiens (A)	240	52,3 %
Copies bruitées (B)	146	31,8 %
Total train	459	s.o.
Test (100% réel, verrouillé)	19	s.o.

Le jeu de test reste **100 % réel** tout au long de l’expérience : aucune donnée synthétique n’y entre.

La figure 4.6 montre que les distributions synthétiques (vert) épousent bien la forme des données réelles (bleu) sur les 6 features représentées ; la gaussienne ne génère pas des profils aberrants.

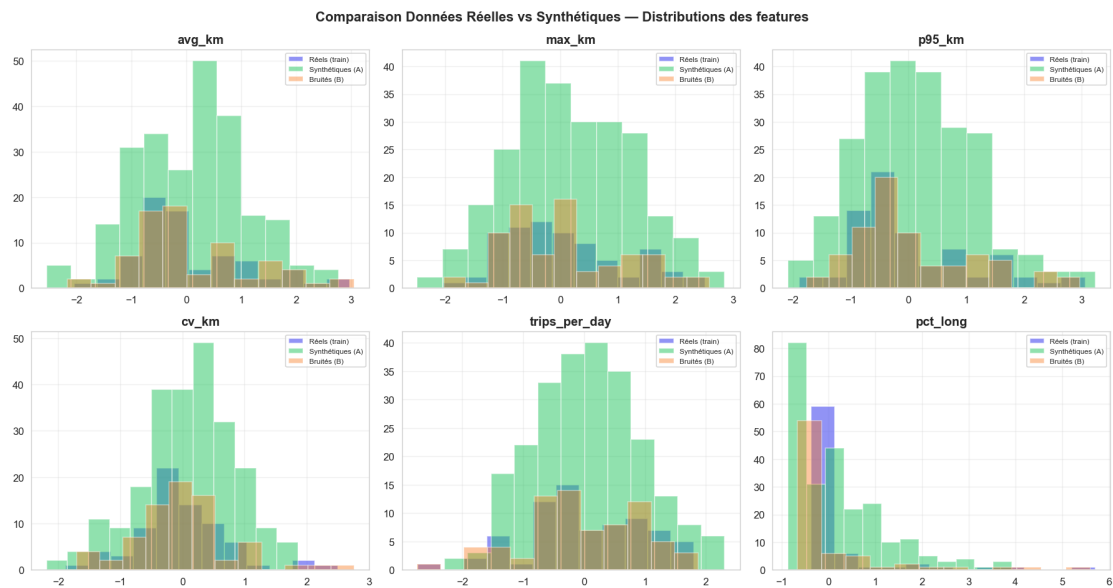


FIGURE 4.6 – Comparaison données réelles vs synthétiques : distributions normalisées pour 6 features clés. Les synthétiques (vert) suivent la même forme que les réels (bleu), ce qui valide l’hypothèse gaussienne par classe.

4.7.3 Résultats

La figure 4.7 résume les résultats : matrice de confusion, courbe ROC et accuracy par fold en cross-validation.

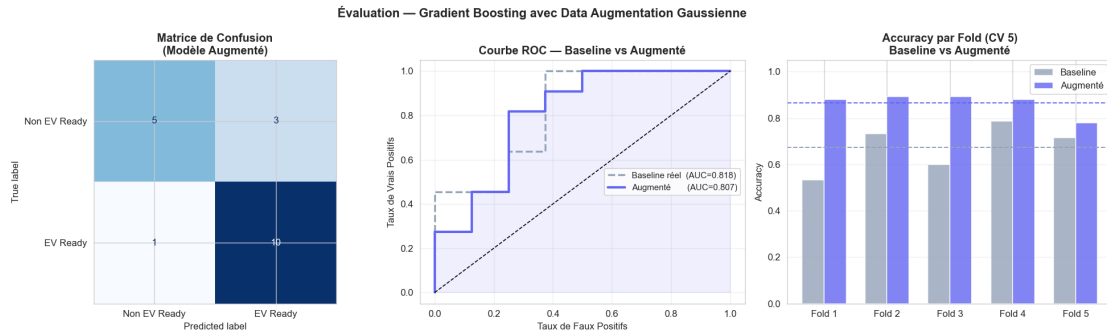


FIGURE 4.7 — Évaluation du modèle augmenté : matrice de confusion (gauche), courbe ROC baseline vs augmenté (centre), accuracy par fold CV-5 (droite). L’augmentation stabilise fortement la variance inter-fold.

L’augmentation réduit l’instabilité : la variance CV passe de $\pm 9,3\%$ à $\pm 4,3\%$, l’écart train/test est divisé par 2,4. Sur le holdout réel, les deux modèles donnent la même accuracy (78,9 %), ce qui confirme que la donnée synthétique n’introduit pas de biais. La matrice de confusion montre que le modèle est meilleur sur les EV Ready (rappel 91 %) que sur les Non EV Ready (rappel 62 %) ; les faux négatifs sont peu nombreux, ce qui est le bon compromis pour ce cas d’usage (mieux valoir convertir un conducteur de plus que d’en rater un).

4.7.4 Concordance avec le score métier

L’analyse des importances de features est détaillée en section 4.2. Pour la validation finale, le modèle est appliqué aux 92 conducteurs réels : le Gradient Boosting et le scoring métier sont d’accord sur **94,6 % des cas** (87/92). Les 5 désaccords concernent des profils à la frontière entre deux classes, là où les deux approches ont légitimement des marges d’incertitude. Ce niveau de concordance valide que les coefficients calibrés manuellement capturent bien les structures présentes dans les données, et que les douze features retenues portent effectivement l’information discriminante.

Conclusion

Au final, l'outil est en production et les équipes de 3C l'utilisent, ce qui n'était pas acquis d'avance. En apparence la question de départ était simple : exploiter les données GPS que Masternaut collectait déjà, pour faire des analyses que Masternaut ne faisait pas. En pratique ça a touché à des sujets assez différents : performance front-end sur Leaflet, estimation carburant depuis des événements d'arrêt moteur tournant, clustering spatial sur du GPS brut, et classification ML avec un dataset trop petit pour être confortable.

Chaque module répond à quelque chose de concret. La carte donne pour la première fois une vue trajet par trajet d'une journée de technicien, avec les temps d'arrêt, la sélection d'un trajet unique isolé et les bornes IRVE directement comparées aux distances d'arrêt. Le module arrêt moteur tournant a chiffré quelque chose que personne n'avait pu quantifier avant : HJ477BP seul cumule plus de 8 heures d'arrêt moteur tournant et environ 7,4 L gaspillés sur un mois, sur 72 véhicules actifs. Le dashboard comportemental donne à l'équipe technique un classement pour cibler les formations éco-conduite, avec un scoring qui tient compte de la distance réellement parcourue. Et le scoring EV remplace le raisonnement à l'intuition par une priorisation basée sur les données réelles de chaque conducteur.

Il y a quelques limites à reconnaître. L'estimation carburant du module arrêt moteur tournant a une incertitude d'environ $\pm 20\%$: la consommation à l'arrêt moteur tournant dépend du régime moteur, de la température, de l'état du véhicule, autant de paramètres qu'on n'a pas. C'est suffisant pour identifier les véhicules problématiques, pas pour sortir une facture. Sur le scoring EV, 92 conducteurs c'est petit : la data augmentation a aidé à stabiliser le modèle, mais avec plus de données on affinerait mieux les cas intermédiaires entre *EV Faisable* et *Hybride recommandé*. La base IRVE est imparfaite aussi : une borne dans la base peut être hors service, une borne récente peut ne pas encore y figurer. Et sans backend, si Masternaut change ses noms de champs (ce qui s'est déjà produit pendant le projet), il faut aller corriger à la main dans le code.

Quelques pistes d'amélioration sont assez évidentes. La plus utile serait d'alimenter le modèle au fil des conversions réelles : quand un conducteur passe effectivement à

l'électrique, son profil GPS avant conversion devient une donnée d'entraînement bien meilleure qu'un profil synthétique. Le réseau IRVE évolue aussi vite en ce moment que relancer le scoring tous les 3 à 6 mois permettrait de mettre à jour le plan de renouvellement automatiquement. Et même si l'outil a été développé pour 3C, le pipeline DBSCAN + scoring EV est suffisamment générique pour s'adapter à n'importe quelle flotte avec des boîtiers GPS, moyennant quelques ajustements de configuration.

Remerciements

Je tiens à remercier l'équipe IT de 3C qui m'a accompagné tout au long de cette première vraie expérience professionnelle.

Un remerciement particulier à **Jean-Philippe Edom**, tuteur entreprise, qui malgré un planning chargé a toujours trouvé le temps de m'apprendre et de m'expliquer les choses avec pédagogie. Sa disponibilité et sa bienveillance ont été déterminantes dans ma progression.

Je remercie également **Eric Nhouyvanisvong**, qui a été mon mentor lors de ma première année et m'a transmis de nombreuses compétences. Son accompagnement a posé des bases solides sur lesquelles j'ai pu m'appuyer tout au long de cette alternance.

Je tiens à remercier tout particulièrement **Badr Tajini**, mon tuteur pédagogique à l'EFREI, qui m'a accompagnée pendant ces deux années de cycle ingénieur. C'est lui qui m'a suggéré d'utiliser la data augmentation pour contourner la limite des 92 conducteurs analysables, une intuition qui s'est révélée déterminante pour la stabilité du modèle. Sa disponibilité et sa rigueur scientifique ont profondément marqué ma manière d'aborder un problème de Data Science.

Trois années à l'EFREI auront transformé une curiosité pour les chiffres en une conviction : la donnée n'a de valeur que lorsqu'elle éclaire une décision, et c'est ce métier que je m'appête à exercer.